

Q-STR: Diagnosing Query-Driven Text-to-Table Reasoning in Cricket Narratives

Ritam Upadhyay^{1*} Naman Ahuja^{1*} Rishabh Baral¹ Aparna Garimella² Vivek Gupta^{†1}

¹School of Computing and Augmented Intelligence, Arizona State University

²Adobe Research, India



Abstract

Many analytical questions over long narratives are structured even when their source documents are not. Answering them requires identifying relevant events, interpreting a query-specific output structure, grounding answer records in dispersed evidence, and computing the values of an answer table. We introduce Q-STR, a cricket-domain benchmark and diagnostic for this query-driven text-to-table problem. We construct Q-STR from ball-by-ball match narratives using executable SQL templates and release two complementary splits: a 4,256-example coverage split for breadth and a 3,467-example controlled split spanning 348 reasoning templates for within-template diagnosis. Factorized metrics distinguish recovery of the answer-table structure from fidelity of its cell values. Across four language models and seven prompting strategies, models often recover plausible table structure while failing at grounded aggregation. Gemini-2.5-Flash is strongest overall, but even its best configuration reaches only 28.56 exact table match. For open-weight models, allocating output tokens to row-local evidence and computation improves cell fidelity more reliably than multi-call planning or self-consistency. Controlled comparisons further show that event identification and numeric fidelity are only partially coupled: execution pressure is the most consistent difficulty driver, while increasing output pressure shifts errors toward undercounting. Q-STR is deliberately a cricket diagnostic rather than a general benchmark of all narrative reasoning. Its single-domain scope, model-assisted construction, and absence of an executable code-generation baseline limit broader conclusions, but its controlled design exposes where structured reasoning from long narratives breaks down.

1 Introduction

Many real-world questions require more than retrieving a short span from a long document. Live commentary, event logs, case notes, and operational reports distribute evidence across local updates while the underlying state changes over time (Kočiský et al., 2018; Dua et al., 2019; Pang et al., 2022; Bertsch et al., 2025). A useful answer may require identifying a set of events, resolving references to evolving entities, aggregating information across distant passages, and presenting the result in a structure that supports inspection. The necessary facts may all be present in the text, yet the organization required by the question may be absent.

We study this setting as *query-driven text-to-table generation*. Given a natural-language query and a long narrative document, a model must produce an answer table directly from textual evidence. The goal is not to recover one canonical table for the document. Instead, each question defines a task-specific analytical view of the narrative. In Q-STR, the requested output columns and primary-key metadata are made explicit to the model, so the structural challenge is not unconstrained schema discovery. The model must interpret this query-conditioned output specification, determine which entities or event groups instantiate its rows, identify the relevant evidence, and compute the requested values. A question asking for wickets by bowler and batter requires a different organization from one asking for boundary-run percentages or strike rates over a temporal interval, even when both questions refer to the same match.

This formulation couples three abilities. First, the model must interpret the supplied query-conditioned output specification and determine which entities or event groups instantiate its rows. Second, it must ground those rows and their values in narrative evidence. Third, it must execute the required filters, groupings, comparisons, and

*These authors contributed equally

[†]Primary supervisor for this work

arithmetic. A correct-looking table can fail at any of these stages: it may contain the wrong entities, the right entities with unsupported values, or correctly extracted values computed over the wrong time window. Final-answer exact match alone does not reveal which capability failed.

We introduce Q-STR, a benchmark for query-driven table generation from cricket commentary. Cricket is a productive diagnostic domain because its narratives are long, sequential, and stateful. Relevant evidence is distributed across many short updates; player mentions depend on the current innings and matchup; and analytical questions naturally involve filtering, grouping, comparison, temporal restriction, and numeric aggregation. At the same time, the underlying ball-by-ball state is structured enough to support executable gold answers and controlled evaluation. This combination lets us study the “missing table” problem in a setting where the latent records are auditable.

Q-STR contains two complementary splits. The *coverage split* measures breadth across query structures and match contexts. The *controlled split* fixes a reasoning template while varying grounded inputs such as player identities, matches, and temporal conditions. The latter supports paired and pressure-based analyses that are difficult to conduct on a random benchmark split. We evaluate table-level correctness, primary-key recovery, aligned cell accuracy, numeric error, and directional counting bias. Together, these metrics separate event identification from value computation.

Our experiments compare four language models and seven prompting strategies. The coverage split produces a clear model ranking, but also a consistent gap between recovering rows and filling them correctly. Decomposition and self-consistency can stabilize table structure without fixing correlated arithmetic errors. Row-local reasoning performs better at cell-level computation for the open-weight models despite using fewer total tokens than several multi-call methods. On the controlled split, Llama-3.3-70B-Instruct and Qwen2.5-72B-Instruct have nearly identical PK F1 but different precision–recall profiles. Their numeric behavior also differs: both shift from overcounting to undercounting as output tables become larger, with a sharper shift for Qwen.

These findings support a focused claim. Q-STR does not establish that performance transfers to finance, medicine, law, or other narrative domains. Rather, it provides a controlled cricket testbed in

which schema interpretation, evidence recovery, and numerical execution can be measured separately. The benchmark is useful precisely because its domain permits dense, reproducible diagnosis; broader generality remains an empirical question.

Our contributions are:

- We formulate query-driven text-to-table generation as a joint problem of query-conditioned structure interpretation, long-context evidence grounding, and computation.
- We introduce Q-STR, with a 4,256-example coverage split and a 3,467-example controlled split comprising 348 templates instantiated across five matches.
- We provide factorized table, primary-key, cell, and numeric-bias metrics that distinguish event identification from value fidelity.
- We benchmark four models under seven prompting strategies and show that row-local grounded reasoning improves value fidelity more consistently than planning or repeated generation.
- We use controlled factors and matched pressure analyses to identify execution burden as the most stable difficulty driver and to expose systematic undercounting as answer tables scale.

2 Related Work

Text-to-table generation. The closest line of work converts unstructured text into structured tables, including direct text-to-table generation, order-insensitive generation, commentary-to-table mapping, global tuple extraction, and latent-schema recovery (Wu et al., 2022; Li et al., 2023b; Deng et al., 2024; Ahuja et al., 2025). These tasks support downstream information extraction, summarization, and analysis. Many define a target schema at the dataset or instance level, or ask for a canonical summary of the input. Q-STR instead makes the analytical question part of the structural specification: the same narrative can support many different answer tables. The requested output columns and primary-key metadata are supplied to the model, but it must still determine which narrative entities instantiate the requested rows and compute the query-specific values.

Text-to-SQL and semantic parsing. Text-to-SQL maps natural-language questions to executable programs over structured databases (Pasupat and Liang, 2015; Yu et al., 2018; Wang et al.,

2020; Scholak et al., 2021; Li et al., 2023a; Lei et al., 2025). This literature provides useful abstractions for filtering, grouping, aggregation, comparison, and compositional reasoning under schema constraints. Q-STR reverses an important assumption. At test time there is no database interface or supplied table over which to execute the query. The system must recover the relevant records from narrative evidence before it can perform database-like reasoning. Gold SQL is used to construct and validate the benchmark, but is not exposed to the evaluated models.

Long-context and numerical question answering. NarrativeQA and QuALITY study question answering over long documents, while DROP targets discrete reasoning over paragraphs (Kočiský et al., 2018; Pang et al., 2022; Dua et al., 2019). Oolong further demonstrates that aggregation across many segments remains difficult even for strong long-context models (Bertsch et al., 2025). Financial benchmarks such as FinQA combine report text, tables, and numerical reasoning (Chen et al., 2021). These settings establish the difficulty of evidence localization, reference resolution, and multi-step computation. Q-STR adds query-dependent structured output: models must recover not only a scalar answer, but also the rows and cells that constitute it.

Controlled and model-assisted benchmark construction. SCAN and CFQ motivate controlled splits for testing systematic generalization under structural variation (Lake and Baroni, 2018; Keysers et al., 2020). StrategyQA and DROP show the value of datasets organized around particular reasoning demands (Geva et al., 2021; Dua et al., 2019). Self-Instruct demonstrates the scalability of model-assisted instruction generation, while StructText uses execution-grounded synthetic supervision for structured generation (Wang et al., 2023; Kashyap et al., 2025). Q-STR combines human-authored executable seeds, model-assisted intent expansion, execution filtering, consistency checks, and human validation. This pipeline scales analytical coverage, but it also makes construction provenance an important part of interpreting the benchmark.

3 Task Formulation

Let Q be a natural-language query and D a narrative document. The task is to generate an an-

swer table $\hat{T}$ that matches a gold table T^* . A table contains an ordered column set $C(T)$ and a set or sequence of rows $R(T)$. When defined by the query, one or more columns form a primary key that identifies the answer records.

Unlike conventional table question answering, Q-STR does not provide T^* , its rows, or a database containing the relevant events. The model receives the question, the requested output columns, primary-key metadata when available, and the complete commentary. Thus, Q-STR does not require unconstrained discovery of column names or key structure. We use *schema interpretation* to refer to understanding how the supplied query-conditioned columns and keys organize the answer and which narrative entities instantiate its rows. The output table must recover both the row structure and the values required by the query. We view this as three coupled decisions:

1. **Schema interpretation:** interpret the supplied target columns and key metadata and determine how they organize the requested answer.
2. **Event identification:** locate the entities or event groups in the narrative that instantiate the answer rows.
3. **Value computation:** apply the query’s filters, aggregations, comparisons, temporal restrictions, or arithmetic to populate non-key cells.

The distinction between the latter two stages is central. Consider a question asking for each batter’s percentage of runs against a bowler that came from boundaries. A model may correctly identify every qualifying batter but miscount total runs, or it may compute a percentage correctly for only a subset of the required batters. Exact table match treats both predictions as failures. Primary-key metrics reveal whether the answer entities were recovered, while aligned cell metrics reveal whether the corresponding values were computed correctly.

4 Benchmark Construction

4.1 Source Narratives and Intermediate Tables

We construct Q-STR from public ball-by-ball cricket commentary covering 620 matches. Cricket commentary is a sequence of local event descriptions, often interleaved with match-state summaries and prose analysis. Following Upadhyay et al. (2026), we first convert each match narrative into a

structured ball-by-ball table. This intermediate representation records fields such as innings, over, batter, bowler, runs, extras, and wickets. It is used only to generate and validate benchmark instances; evaluated systems receive the original narrative rather than this table.

Figure 1 summarizes the pipeline. Executable SQL templates define analytical operations over the intermediate tables. Instantiating a template on a match produces both a natural-language question and a gold answer table. Execution makes complex multi-row answers reproducible and enables automatic removal of invalid or uninformative candidates.

4.2 Agentic Query Synthesis

The query bank begins with two sources. First, we author approximately 120 dataset-specific seed queries, each paired with an executable SQL program and a natural-language question. These seeds establish grounded examples of the analytical behavior the benchmark should contain. Second, we collect approximately 1,000 intent-level ideas with GPT-5.2 to broaden the space of goals, including player comparisons, temporal windows, matchup analysis, conditional aggregation, and streak computation.

Given the table schema, a seed, and a sampled intent, Gemini-3-Flash proposes a new analytical intent, a query structure, and an executable SQL template with placeholders for entities and conditions. Candidate templates are canonicalized and deduplicated, instantiated with grounded variables, and executed over the intermediate match tables. Programs that fail, return empty results, or produce uninformative outputs are removed. The pipeline then generates a corresponding natural-language question and meaning-preserving paraphrases. Each retained instance therefore links a narrative, an executable program, a requested output specification, and a gold answer table.

Model assistance is substantial rather than incidental. Roughly half of the released questions pass through model-assisted rewriting or repair. This increases surface variation and makes the construction pipeline scalable, but can introduce stylistic regularities associated with the generator models. We retain provenance flags for human-authored, model-generated, rewritten, and repaired items so that performance can be stratified by construction path.

4.3 Validation

Syntactically valid SQL does not guarantee that its natural-language question expresses the same operation. We therefore run a text-to-SQL consistency check for every question–program pair. A model regenerates SQL from the natural-language question, and the original and regenerated programs are executed on the same match table. If their outputs agree, the item proceeds. When they disagree, an LLM adjudicator determines whether the original program, regenerated program, or neither correctly represents the question. Corrected pairs must pass the same execution check before retention.

Among disputed cases, the adjudicator selected the original SQL in 511 instances, selected regenerated SQL in 590, and required a corrected pair in 944. The authors additionally reviewed a representative sample of 400 questions and found 97.2% semantic coherence between the natural-language query and gold SQL. This audit measures program–question agreement, not the naturalness of the question distribution; the latter remains a limitation of assisted construction.

4.4 Benchmark Splits

The **coverage split** contains 4,256 question–answer pairs and is designed for breadth. It varies both query structure and match context, making it suitable for overall model and prompting comparisons.

The **controlled split** contains 3,467 question–answer pairs generated from 348 unique templates. Each template has roughly ten instantiations distributed across five matches. The reasoning operation remains fixed while grounded variables, such as player names or temporal boundaries, vary. This supports within-template comparisons and reduces the extent to which diagnostic effects can be attributed to unrelated differences in query form.

The coverage and controlled sets are complementary evaluation sets rather than train/test partitions. The coverage split is designed to maximize breadth across query structures and match contexts, whereas the controlled split intentionally repeats reasoning templates under different grounded instantiations to support matched diagnosis. All examples originate from the same cricket-commentary source distribution and may share domain-specific discourse regularities.

The use of a small fixed match set in the controlled split is intentional: repeatedly instantiating the same reasoning template under a limited set

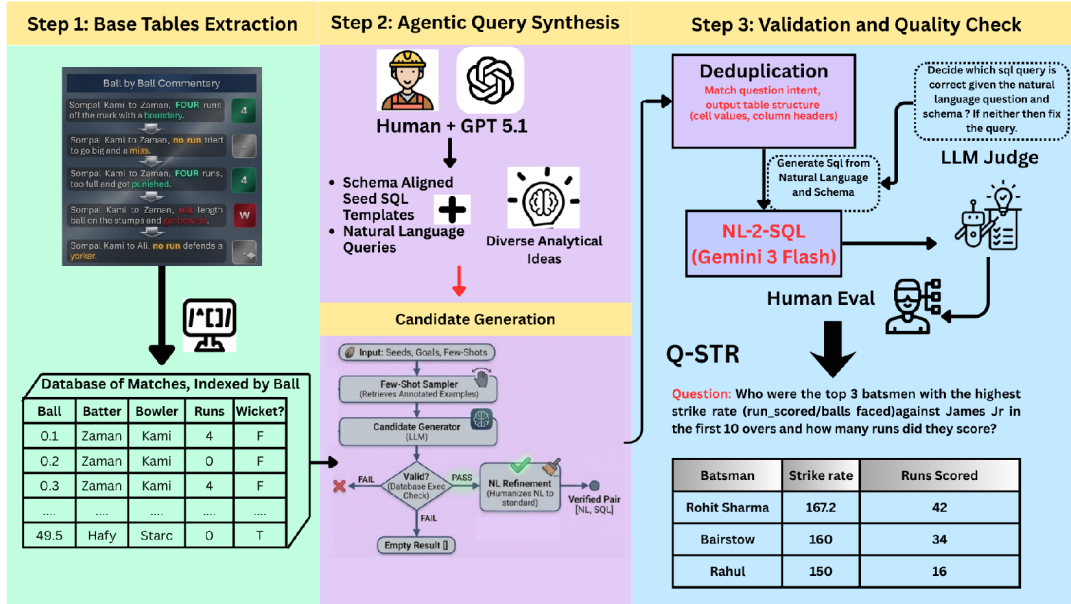


Figure 1: The Q-STR construction pipeline. Ball-by-ball events from 620 games are converted to one intermediate table per match. Human-authored seeds and model-generated analytical intents produce candidate SQL and natural-language pairs, which are execution-filtered, checked for cross-modal consistency, and reviewed before entering the benchmark.

of match contexts reduces variation in query structure and supports matched within-template comparisons. This improves diagnostic control but reduces match-level diversity, so the controlled split should not be interpreted as an estimate of performance across the full distribution of cricket matches.

Controlled examples are annotated along five skill families: entity and schema grounding, context scoping, temporal reasoning, query operations, and compositional depth. The query-operation labels are simple retrieval, filtered aggregation, grouped aggregation, and group comparison. We additionally record answer width and height. Figure 2 shows the distribution of the controlled annotations; Appendix D gives exact definitions and examples. Category-level behavior is examined on the controlled split through factor contrasts over temporal reasoning, entity grounding, operation class, and compositional depth in Section 7 and Appendix E.

4.5 Benchmark Characterization

Table 1 summarizes the split statistics verified from the current manuscript and repository.

Statistic	Coverage	Controlled
Examples	4,256	3,467
Reasoning templates	—	348
Source matches represented	—	5

Table 1: Summary statistics for the Q-STR evaluation splits. Dashes indicate statistics not defined in the same way for that split in the current manuscript.

5 Evaluation

5.1 Factorized Metrics

We report metrics at three levels. **Table-level metrics** measure end-to-end correctness. Exact Table Match requires normalized columns, rows, and values to match the reference. Row Match compares complete row signatures and is insensitive to row ordering after canonicalization.

Event-identification metrics evaluate the primary-key tuples that define answer rows. We report PK Exact, precision, recall, and F1. These metrics reveal whether a model found the correct entities or event groups even when non-key values are wrong.

Value-fidelity metrics align predicted and gold rows by primary key. Cell Accuracy measures exact non-key cell agreement, using a small tolerance for numeric normalization. Weighted RMSE aggregates column-wise numeric error while accounting

Type	Representative question
Filtered aggregation	How many deliveries did Verreynne face while batting against Hasan Ali?
Temporal filtering	List wickets claimed by each bowler against each batter before over 29.5.
Grouped aggregation	For each batter who scored off Faheem, what percentage of their runs against him came from boundaries?
Group comparison	Compare each batter’s strike rate against Shaheen Shah Afridi on boundary and non-boundary deliveries.
Streak reasoning	For each batter, what is the longest streak of consecutive dot balls against Faheem?

Table 2: Representative Q-STR question and operation types. Names and temporal values are grounded variables that change across instantiations.

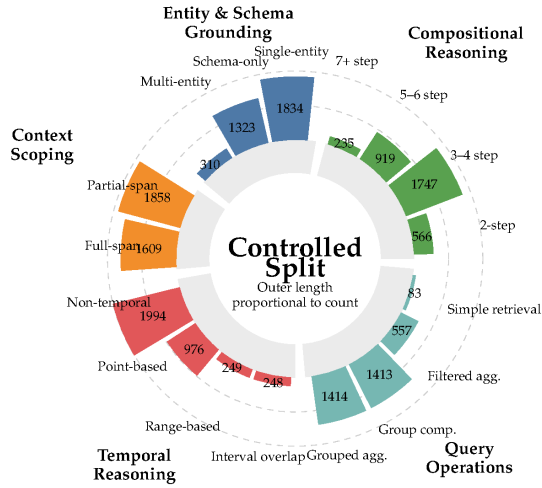


Figure 2: Distribution of controlled-split examples across five skill families. Outer radius is proportional to the count of each subskill, normalized by the largest category.

for the number of aligned cells in each column. On the controlled split we also report Overcount%, Undercount%, and their difference, Net Bias. Exact definitions appear in Appendix A.

Semantic Text-to-Table evaluation metrics are useful when multiple surface realizations may express equivalent content. Q-STR instead uses executable SQL-derived references with explicit row keys and numerically precise values. In this setting, selecting an extra entity, omitting a qualifying row, or producing an incorrect aggregate is a substantive reasoning error rather than a surface-form mismatch. We therefore prioritize exact and factorized metrics that separately evaluate table recovery, primary-key recovery, aligned cell correctness, and

numerical error. Canonicalization and numeric tolerance absorb superficial formatting differences. Semantic metrics are complementary, but less directly aligned with Q-STR’s goal of diagnosing exact event recovery and numerical execution.

5.2 Prompting Baselines

All baselines receive the same question, requested output columns, primary-key metadata, and full commentary, but allocate test-time computation differently:

- ZERO SHOT directly generates the final answer table without visible intermediate reasoning.
- ZERO-SHOT COT uses one call with a free-form reasoning trace before table generation.
- GUIDED COT imposes three stages within one call: identify entities, compute statistics, and construct the table.
- PLANFILL first predicts row headers and a short plan, then fills values in a second call.
- SC TABLE VOTING samples five complete solutions, aligns rows by primary key, and votes over row structure and cell values.
- RowCOT produces one evidence-and-computation trace per output row within a single call.
- LEAST-TO-MOST generates a directed acyclic graph of at most five subproblems and executes them in separate calls.

These baselines test direct generation, visible reasoning, structure-first decomposition, repeated sampling, and row-local computation. They do not include model-generated code or SQL over the intermediate event tables. Such a privileged-structure baseline would separate program induction from narrative extraction, but its empirical comparison requires an additional experiment; we return to this issue in Section 9.

5.3 Models and Inference

We evaluate Llama-3.3-70B-Instruct, Qwen2.5-72B-Instruct, GPT-4.1, and Gemini-2.5-Flash. The full seven-baseline suite is run on Llama and Qwen. Because of compute constraints, GPT-4.1 and Gemini are evaluated with ZERO SHOT and GUIDED COT. Open-weight models are served with vLLM

on two H200 nodes with 80GB GPUs each. GPT-4.1 uses the OpenAI Batch API, and Gemini uses the Google AI Studio API.

Most methods use deterministic or near-deterministic decoding. SC TABLE VOTING uses temperature 0.6 to create candidate diversity. Outputs are parsed as strict JSON and checked for exact column order, row-length consistency, and method-specific fields. Appendix C provides prompt contracts, inference budgets, and aggregation details.

6 Coverage-Split Results

Table 3 reports the breadth-oriented benchmark results. Exact table match is deliberately strict and should be interpreted alongside row, key, and cell metrics.

6.1 Model Capability Dominates Overall Ranking

Gemini-2.5-Flash is strongest by a large margin, followed by GPT-4.1, Llama, and Qwen. This ordering is stable across the two baselines shared by all four models. Under ZERO SHOT, Gemini reaches 94.51 PK F1, 71.52 Cell Accuracy, and 7.28 RMSE, compared with GPT-4.1 at 91.30/40.96/14.09, Llama at 88.40/31.99/20.42, and Qwen at 87.66/30.16/21.29. Guided reasoning improves GPT-4.1 substantially and lowers Gemini’s RMSE, although Gemini’s exact match and Cell Accuracy do not increase.

The absolute gap between row recovery and exact table match is important. Even Gemini’s 94.58 PK F1 under GUIDED CoT corresponds to only 24.83 exact table match. Models frequently identify the relevant answer entities without recovering every required value. This supports evaluating structure and cells separately rather than treating table generation as a monolithic output-format problem.

6.2 Where Additional Output Tokens Help

For Llama and Qwen, adding reasoning within one call generally improves cell-level fidelity. Moving from direct generation to free-form, guided, or row-local reasoning increases Cell Accuracy and usually lowers RMSE. ROWCoT is the clearest case: it obtains the best Cell Accuracy and RMSE for Llama and the best Cell Accuracy for Qwen. Its per-row traces attach local evidence and arithmetic to each predicted record, reducing the burden

of maintaining one global computation across a multi-row answer.

Total token expenditure alone does not explain the gain. PLANFILL, SC TABLE VOTING, and LEAST-TO-MOST use multiple calls and substantially more total tokens, but they do not consistently outperform the strongest single-call baselines on value metrics. Additional computation is most useful when it directly supports evidence extraction and cell calculation rather than verbose planning or repeated whole-table generation.

6.3 Decomposition Helps Structure More Than Values

SC TABLE VOTING and LEAST-TO-MOST sometimes improve Row Match, PK Exact, or PK F1, but these gains do not translate consistently to Cell Accuracy or RMSE. Self-consistency stabilizes which rows appear, yet candidates often share the same computational mistake. On Llama, the voted table has average cell support of 3.83 out of five candidates, and 85.9% of samples have average support above three, but its RMSE remains worse than ZERO-SHOT CoT. Agreement is therefore correlated without necessarily being corrective.

PLANFILL is a particularly informative negative result. It often has high PK recall, suggesting that the planner proposes plausible entities, but low precision and Row Match indicate that the proposed structure contains spurious or mismatched records. The second stage then inherits this structure and still must solve the harder cell-computation problem. Separating row planning from value filling does not by itself create a reliable intermediate representation.

7 Controlled-Split Analysis

The coverage split establishes broad performance. The controlled split is designed for diagnosis. Because it concentrates repeated template instantiations within five matches, its contrasts are intended to diagnose failure modes under matched conditions rather than estimate population-wide cricket performance. The restricted match set reduces variation in narrative context but may also couple observed effects to the styles and event distributions of those matches. We analyze Llama and Qwen because the full baseline suite is available for both, and focus on their ZERO SHOT outputs to avoid conflating the underlying factor analysis with baseline-specific scaffolds. Each controlled tem-

Method	Exact Table	Row Match	PK Exact	PK Prec.	PK Rec.	PK F1	Cell Acc.	RMSE ↓
Llama-3.3-70B-Instruct								
ZERO SHOT	2.69	50.52	48.42	83.33	94.13	88.40	31.99	20.42
ZERO-SHOT CoT	5.38	57.03	55.09	87.09	94.34	90.57	37.26	17.67
GUIDED CoT	4.34	52.26	50.23	79.68	96.82	87.42	37.16	17.01
PLANFILL	3.65	41.15	38.97	73.43	97.19	83.65	34.33	18.89
SC TABLE VOTING	5.47	58.42	56.53	88.34	93.79	90.98	36.58	18.12
ROWCoT	3.30	52.43	50.32	82.11	95.00	88.09	38.98	16.16
LEAST-TO-MOST	5.30	52.78	50.95	81.83	91.27	86.29	35.02	17.66
Qwen2.5-72B-Instruct								
ZERO SHOT	2.26	44.27	41.49	82.84	93.07	87.66	30.16	21.29
ZERO-SHOT CoT	3.04	51.74	49.59	85.31	94.21	89.54	33.71	17.58
GUIDED CoT	2.95	49.91	47.79	80.61	96.80	87.97	35.48	17.60
PLANFILL	3.04	40.10	37.62	81.88	91.53	86.44	33.09	19.47
SC TABLE VOTING	3.21	50.52	48.06	88.36	91.22	89.77	31.47	19.34
ROWCoT	3.39	45.83	43.56	81.64	94.71	87.69	36.63	18.03
LEAST-TO-MOST	4.25	55.90	54.01	84.79	95.45	89.80	33.67	18.28
GPT-4.1								
ZERO SHOT	5.30	64.24	62.74	89.29	93.39	91.30	40.96	14.09
GUIDED CoT	8.51	71.18	69.58	90.72	95.29	92.95	45.97	12.37
Gemini-2.5-Flash								
ZERO SHOT	28.56	80.56	79.21	93.19	95.87	94.51	71.52	7.28
GUIDED CoT	24.83	79.43	78.04	92.69	96.56	94.58	71.18	6.28

Table 3: Performance on the Q-STR coverage split. Exact Table and Row Match measure complete outputs; PK metrics measure answer-row recovery; Cell Accuracy and weighted RMSE measure value fidelity on aligned rows.

plate is instantiated under closely related grounded conditions, allowing paired comparisons across event identification and numeric fidelity.

7.1 Analysis Protocol

We annotate examples with fine-grained factors covering grounding, context scoping, temporal reasoning, evidence sparsity, query-operation class, compositional depth, answer width, and answer height. For summary analysis, these factors are aggregated into four pressure axes:

- **Grounding pressure** captures whether a query requires schema-only, single-entity, or multi-entity grounding.
- **Localization pressure** combines temporal restriction, context span, and evidence sparsity.
- **Execution pressure** combines query-operation complexity with compositional depth.
- **Output pressure** combines the number of answer columns and rows.

Each axis is mapped to low, medium, or high using rules fixed before analysis (Appendix D). These labels summarize observed difficulty conditions; they are not causal variables.

Event metrics are computed over examples with parseable primary keys. Numeric metrics are computed only when predicted and gold rows can be aligned and their values are numerically evaluable. We report paired mean differences with bootstrap confidence intervals and sign-flip permutation tests (Zmigrod et al., 2022; Bestgen, 2022; Clark et al., 2011). Pressure comparisons are stratified by matched conditions where possible. Balanced resampling equalizes support within each contrast and serves as a sensitivity analysis for level imbalance. We call a pressure contrast robust only when its direction is stable across raw, permutation, and balanced analyses.

7.2 Overall Event and Numeric Profiles

Table 4 shows that the models are effectively tied on headline event metrics. PK F1 differs by only −0.1 points and PK Exact by 0.3, with both intervals crossing zero. The clearer difference is an operating trade-off: Qwen has higher recall, while Llama has higher precision. Qwen recovers more gold records but introduces more spurious ones; Llama is more conservative.

Mean weighted RMSE and aligned Cell Accuracy are also close. Directional bias reveals a difference hidden by the aggregate error scores. Llama has the larger positive net bias, while Qwen undercounts more often. Thus, similar mean fidelity can

Stage	Metric	n	Llama	Qwen	Δ_{Q-L}	95% CI
Event identification	PK F1	2551	83.6	83.5	-0.1	[-0.9, 0.8]
	PK Recall	2551	93.8	95.6	1.8*	[1.1, 2.5]
	PK Precision	2551	82.4	80.6	-1.8*	[-2.8, -0.8]
	PK Exact	2551	55.3	55.6	0.3	[-1.4, 1.9]
Numeric fidelity	Weighted RMSE	3129	13.2	12.5	-0.6	[-1.5, 0.3]
	Aligned Cell Accuracy	2345	42.6	42.0	-0.6	[-1.7, 0.7]
	Overcount%	3129	35.4	34.1	-1.3	[-2.8, 0.1]
	Undercount%	3129	24.9	27.6	2.7*	[1.4, 3.9]
	Net Bias	3129	10.4	6.4	-4.0*	[-6.4, -1.5]

Table 4: Paired controlled-split comparison. Positive Δ_{Q-L} favors Qwen for event metrics and Cell Accuracy; negative values favor Qwen for RMSE and absolute positive bias. Asterisks indicate 95% intervals excluding zero.

arise from different error distributions.

7.3 Fine-Grained Factors

Appendix Table 8 gives representative factor contrasts. Temporal filtering primarily stresses event recovery: point-based and range-based conditions reduce Llama’s PK F1 by 8.7 and 9.1 points relative to non-temporal controls. Multi-entity grounding has a larger and more consistent effect on numeric fidelity, adding roughly 12 RMSE points for both models. Deeper 5–6 step compositions also increase numeric error, although Qwen’s interval crosses zero. These distinct patterns motivate the broader pressure axes.

7.4 Event Identification Under Pressure

Because pooled effects can reflect unequal mixtures of examples, Appendix Table 9 reports the stratified event contrasts whose direction remains stable under permutation testing and balanced re-sampling. Three patterns hold for both models. Medium grounding is easier than low grounding after matching, high execution pressure is harder than medium, and medium output pressure is harder than low. The first pattern is counterintuitive in isolation and likely reflects residual composition in the schema-only group; it should not be interpreted as a causal benefit of entity grounding.

Execution and output effects are more direct. Moving from medium to high execution pressure lowers PK F1 by 4.5 points for Llama and 6.9 for Qwen. Moving from low to medium output pressure lowers it by 12.1 and 9.4 points. Balanced estimates remain similar or larger, suggesting that these effects are not produced only by support imbalance.

7.5 Numeric Fidelity and Bias

Numeric pressure effects are less uniformly separated because weighted RMSE is conditional on parseable, aligned outputs. Appendix Table 10 shows the strongest stratified contrasts. Execution high versus low is robust for both models: weighted RMSE increases by 29.9 for Llama and 43.7 for Qwen, with balanced estimates of 28.3 and 38.3. This is the most consistent difficulty signal in the controlled analysis. Llama also shows a robust grounding high-versus-medium effect. Qwen’s grounding and output effects become visible after balancing but are not supported by the raw permutation tests, so we treat them as sensitivity results.

Directional error adds information beyond RMSE (Appendix Figure 3). At low output pressure, both models overcount. At high pressure, Llama’s net bias is -7.1 , while Qwen’s reaches -24.9 . Larger answer structures therefore appear to induce omission-type errors, especially for Qwen. This result is conditional on outputs that remain evaluable, so it should be read as an error-profile diagnostic rather than an unconditional failure rate.

8 Conclusion

We presented Q-STR, a cricket-domain benchmark for generating query-specific tables from long narratives. Across four models, systems recover primary keys more reliably than cell values, and row-local reasoning outperforms multi-call decomposition on value fidelity. Controlled analysis identifies execution pressure and increasing undercount bias as central failure modes. Q-STR therefore diagnoses the missing-table problem, but broader claims require non-cricket domains, code or SQL baselines over privileged structure, and construction-provenance analysis.

9 Limitations

Single-domain scope. Q-STR is built entirely from cricket commentary. Cricket offers long, stateful narratives and executable event structure, but also has specialized vocabulary, repeated discourse patterns, and unusually regular latent records. Models may exploit domain familiarity, and the measured pressures may behave differently in less regular settings. Results should therefore be interpreted as a cricket-domain diagnosis.

No code or SQL generation baseline. The current experiments prompt models to generate answer tables directly from narratives. They do not test model-generated SQL or code over gold event tables, nor code over model-extracted tables. These comparisons are necessary to determine how much difficulty comes from narrative-to-record extraction versus analytical execution after privileged access to structure. A specialized Text-to-Table baseline would also require adaptation because Q-STR conditions generation on an analytical query and evaluates a query-specific answer table rather than extraction of a single canonical table. Gold SQL execution establishes answer correctness by construction, but it is not a model baseline and should not be treated as one.

Model-assisted construction. The query pipeline uses GPT-5.2 and Gemini-3-Flash for intent expansion, paraphrasing, validation, and repair, and roughly half of the examples undergo assisted rewriting or correction. Execution and human audits establish semantic coherence but do not remove distributional artifacts. Generator-specific syntax or lexical preferences may make some questions easier for related models and less representative of naturally occurring user requests. Provenance-stratified reporting and human-only or naturally collected test sets are needed.

Experimental coverage. The full baseline suite is evaluated only on two open-weight models, while closed models use two prompts. Experiments cover prompting rather than fine-tuning, retrieval systems, or tool-augmented agents. The controlled pressure variables are coarse summaries, matched analyses remain observational, and numeric results are conditional on evaluable aligned outputs. Hidden confounders and survivor effects may remain.

Controlled-split diversity. The controlled split instantiates 348 reasoning templates over only five

matches. This design supports repeated within-template comparisons under relatively stable match contexts, but it limits match-level diversity. Consequently, pressure effects may partly reflect the discourse styles, player distributions, or event composition of these matches and should be interpreted as diagnostic rather than population-wide estimates.

10 Ethical Considerations

The source data consists of publicly accessible cricket commentary and professional athlete names. No private or sensitive records are collected. We retain only match-relevant commentary and associated gameplay information. Nevertheless, public accessibility does not eliminate intellectual-property obligations. Any release should distribute derived annotations, gold tables, provenance, and evaluation code under an appropriate research license while directing users to the original source under its governing terms.

Real player identities can create memorization and association effects. Entity substitution and anonymized evaluations can test whether results depend on familiarity with particular players. The benchmark is intended for academic analysis of structured reasoning and should not be used to make claims about player ability, intent, or reputation. Model-generated and model-repaired examples are marked in provenance metadata so downstream users can audit the role of automated construction.

References

- Naman Ahuja, Fenil Bardoliya, Chitta Baral, and Vivek Gupta. 2025. Map&make: Schema guided text to table generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 30249–30262.
- Amanda Bertsch, Adithya Pratapa, Teruko Mitamura, Graham Neubig, and Matthew R. Gormley. 2025. Oolong: Evaluating long context reasoning and aggregation capabilities. *arXiv preprint arXiv:2511.02817*.
- Yves Bestgen. 2022. Please, don’t forget the difference and the confidence interval when seeking for the state-of-the-art status. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 5956–5962.
- Zhiyu Chen, Wenhui Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. 2021. FinQA: A dataset of

- numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*.
- Jonathan H. Clark, Chris Dyer, Alon Lavie, and Noah A. Smith. 2011. Better hypothesis testing for statistical machine translation: Controlling for optimizer instability. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics*, pages 176–181.
- Zheyang Deng, Chunkit Chan, Weiqi Wang, Yuxi Sun, Wei Fan, Tianshi Zheng, Yauwai Yim, and Yangqiu Song. 2024. Text-tuple-table: Towards information integration in text-to-table generation via global tuple extraction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9300–9322.
- Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. 2019. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *Proceedings of NAACL-HLT*, pages 2368–2378.
- Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. 2021. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies. *Transactions of the Association for Computational Linguistics*, 9:346–361.
- Satyananda Kashyap, Sola Shirai, Nandana Mihindukulasooriya, and Horst Samulowitz. 2025. Structtext: A synthetic table-to-text approach for benchmark generation with multi-dimensional evaluation. *arXiv preprint arXiv:2507.21340*.
- Daniel Keysers, Nathanael Schärli, Nathan Scales, Hylke Buisman, Daniel Furrer, Sergii Kashubin, Nikola Momchev, Danila Sinopalnikov, Lukasz Stafiniak, Tibor Tihon, and 1 others. 2020. Measuring compositional generalization: A comprehensive method on realistic data. *International Conference on Learning Representations*.
- Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. 2018. The NarrativeQA reading comprehension challenge. *Transactions of the Association for Computational Linguistics*, 6:317–328.
- Brenden M. Lake and Marco Baroni. 2018. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *Proceedings of the 35th International Conference on Machine Learning*.
- Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, and 1 others. 2025. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. *arXiv preprint arXiv:2411.07763*.
- Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, and 1 others. 2023a. Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *arXiv preprint arXiv:2305.03111*.
- Tong Li, Zhihao Wang, Liangying Shao, Xuling Zheng, Xiaoli Wang, and Jinsong Su. 2023b. A sequence-to-sequence & set model for text-to-table generation. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 5358–5370.
- Richard Yuanzhe Pang, Alicia Parrish, Nitish Joshi, Nikita Nangia, Jason Phang, Angelica Chen, Vishakh Padmakumar, Johnny Ma, Jana Thompson, He He, and Samuel Bowman. 2022. QuALITY: Question answering with long input texts, yes! In *Proceedings of NAACL-HLT*, pages 5336–5358.
- Panupong Pasupat and Percy Liang. 2015. Compositional semantic parsing on semi-structured tables. *Proceedings of ACL-IJCNLP*.
- Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9895–9901.
- Ritam Upadhyay, Naman Ahuja, Rishabh Baral, Aparna Garimella, and Vivek Gupta. 2026. [Moneyball with LLMs: Analyzing tabular summarization in sports narratives](#). In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 41714–41739, San Diego, California, United States. Association for Computational Linguistics.
- Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. RAT-SQL: Relation-aware schema encoding and linking for text-to-sql parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7567–7578.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 13484–13508.
- Xueqing Wu, Jiacheng Zhang, and Hang Li. 2022. Text-to-table: A new way of information extraction. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, pages 2518–2533.
- Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921.

Ran Zmigrod, Tim Vieira, and Ryan Cotterell. 2022. Exact paired-permutation testing for structured test statistics. In *Proceedings of NAACL-HLT*, pages 4894–4902.

A Metric Definitions

Let T^* denote a gold answer table and $\hat{T}$ a predicted table. Let $R(T)$ denote a canonicalized row signature, $C(T)$ its ordered column signature, and G_{PK} and $\hat{G}_{PK}$ the sets of gold and predicted primary-key tuples when key columns are available. Numeric values are normalized to a fixed precision before comparison.

Row and table metrics. Row Match is an exact comparison of canonical row signatures:

$$\text{RowMatch}(T^*, \hat{T}) = \mathbb{1}[R(T^*) = R(\hat{T})]. \quad (1)$$

The reported dataset score is the mean of this indicator. Exact Table Match is one only when the ordered column signatures, row signatures, and all normalized cells match. It is stricter than Row Match because a prediction with the right row identities but one incorrect value is counted as a complete failure.

Primary-key metrics. When primary-key columns are defined,

$$\text{PKExact}(T^*, \hat{T}) = \mathbb{1}[G_{PK} = \hat{G}_{PK}]. \quad (2)$$

Precision and recall are:

$$\text{PKPrec.} = \frac{|G_{PK} \cap \hat{G}_{PK}|}{|\hat{G}_{PK}|}, \quad (3)$$

$$\text{PKRec.} = \frac{|G_{PK} \cap \hat{G}_{PK}|}{|G_{PK}|}. \quad (4)$$

PK F1 is their harmonic mean. Key values are canonicalized before matching; predictions without a parseable key set receive zero for exact recovery.

Cell Accuracy. Predicted and gold rows are aligned through the primary key when possible. Cell Accuracy is the percentage of aligned non-key cells that match. Text cells use canonical exact comparison. Numeric cells permit a small tolerance to absorb formatting differences such as integer versus decimal notation. Missing, unsupported, or unparseable values are incorrect.

Weighted RMSE. Let V_j be the set of aligned numeric gold–prediction pairs in column j , with gold value v and predicted value $\hat{v}$. Column-wise error is:

$$\text{RMSE}_j = \sqrt{\frac{1}{|V_j|} \sum_{(v, \hat{v}) \in V_j} (v - \hat{v})^2}. \quad (5)$$

We combine columns with a count-weighted mean:

$$\text{WeightedRMSE} = \frac{\sum_j |V_j| \text{RMSE}_j}{\sum_j |V_j|}. \quad (6)$$

This weighting prevents a column with few aligned observations from receiving the same influence as a densely populated column. It does not make values with different natural scales fully comparable, which is why pressure-level RMSE comparisons are interpreted cautiously.

Directional bias. For aligned numeric cells, Overcount% is the percentage for which $\hat{v} > v$, and Undercount% is the percentage for which $\hat{v} < v$. We define:

$$\text{NetBias} = \text{Overcount\%} - \text{Undercount\%}. \quad (7)$$

Positive values indicate an overall tendency to overcount and negative values indicate undercounting. Exact matches contribute to neither rate.

B Dataset Generation and Validation

B.1 Intermediate Event Representation

Each source match is represented by a ball-by-ball event table during benchmark construction. A row records the local game state associated with one legal or illegal delivery, including the innings, over and ball index, batting and bowling entities, runs attributed to the batter and extras, total runs, wicket indicators, and dismissal metadata where available. Commentary remains the test-time input. The event table is an authoring and validation device that permits deterministic SQL execution.

Queries are not restricted to selecting existing columns. SQL templates may compute counts, sums, averages, percentages, rates, extrema, streaks, conditional comparisons, and grouped results. Some questions restrict a span of the match; others compare entities or interactions across the full narrative. The requested answer columns and primary keys are stored with each example so that evaluation can distinguish structural from value errors.

B.2 Generation Stages

The synthesis process proceeds in four stages:

1. **Intent proposal.** A seed query and sampled analytical idea are combined to propose a new goal that remains answerable from the event schema.
2. **Program synthesis.** The goal is translated into parameterized SQL. Placeholders mark variables such as batter, bowler, innings, or temporal boundaries.
3. **Instantiation and execution.** Variables are grounded in actual match entities and valid ranges. The resulting program is executed; failing, empty, degenerate, or duplicate outputs are discarded.
4. **Question realization.** A natural-language question and paraphrases are produced from the accepted program. The wording must preserve conditions, grouping semantics, and requested output fields.

Canonicalization removes superficial SQL differences before deduplication. Execution filtering also excludes templates that always return the same trivial answer across candidate matches. This produces a query bank with controlled variation over entities, attributes, temporal context, and operations.

B.3 Consistency Adjudication

For every natural-language question, a separate model generates SQL from the question alone. We execute this regenerated program and compare its output with the original gold program. Agreement provides an execution-level consistency check. Disagreement triggers adjudication:

- the original SQL is retained when it matches the natural-language intent;
- the regenerated SQL replaces it when the regeneration better expresses the question;
- a corrected question–SQL pair is produced when neither is coherent.

The adjudicator chose the original program for 511 disputed examples, the regenerated program for 590, and a corrected pair for 944. Corrected items are re-executed and must pass the same consistency check. A manual audit of 400 items found 97.2% question–program coherence. The release records whether an instance was accepted directly, adjudicated, or repaired.

C Baseline Prompting and Inference

C.1 Common Input and Output Contract

All methods receive the same core fields: the natural-language question, target columns in their required order, row-key columns, primary-key metadata, and complete ball-by-ball commentary. A common policy block defines how to interpret runs, balls, wickets, overs, rates, and percentages. The commentary is designated as the source of truth. Models are instructed to return strict JSON, preserve target columns exactly, place computed numbers rather than expressions in numeric cells, and use null rather than inventing unsupported values.

The canonical final output is:

```
{
  "columns": ["<col 1>", "<col 2>"],
  "rows": [[<r1c1>, <r1c2>], ...]
}
```

Methods with visible reasoning wrap this object in a method-specific field. The evaluator extracts only the final table for scoring.

C.2 Prompting Methods

ZERO SHOT. One call directly produces the final table. The system prompt asks the model to reason internally, but the visible output may contain only the JSON table. This is the minimal end-to-end baseline and provides no explicit mechanism for inspecting intermediate decisions.

ZERO-SHOT CoT. One call returns a free-form reasoning_trace followed by final_table. The trace must be instance-specific and may include relevant entities, filters, intermediate counts, calculations, and row decisions. This tests whether visible unconstrained reasoning improves over direct generation.

GUIDED CoT. One call must produce three ordered reasoning records: identify relevant entities, compute required statistics, and build the final table. The fixed scaffold is the only difference from free-form chain-of-thought. It tests whether explicitly separating grounding, calculation, and assembly helps within a single context window.

PLANFILL. The planner receives the full instance but is prohibited from computing final values. It returns row headers expressed through the supplied key columns and a concise operational plan. The second call receives the original instance,

planner JSON, and an empty table skeleton. It fills values while preserving the proposed row structure unless the commentary clearly requires correction. This baseline isolates a structure-first decomposition.

SC TABLE VOTING. Five candidates are independently sampled from the same prompt. Each contains a reasoning trace and complete table. Primary-key rows are aligned across candidates using fuzzy key matching and Hungarian assignment. Candidate tables first vote over row signatures. Majority-supported extra rows can be retained. Cells then vote within aligned rows using canonical exact matching; numeric values within tolerance 0.2 are clustered. Without a usable key, aggregation falls back to row order.

ROWCOT. One call produces a non-empty `row_reasoning` list and a final table. Each item contains one row-key object and a trace explaining how that row’s values were derived. The output contract requires exactly one reasoning item per final row. This allocates visible computation locally rather than asking one trace to maintain state for the whole table.

LEAST-TO-MOST. A planner first generates a directed acyclic graph of at most five steps. Each step declares dependencies, whether it needs the full commentary, which prior output keys it consumes, and what structured key it produces. Independent steps can execute in parallel. Executor calls solve each subproblem, passing structured outputs to dependent steps. Only the final step may emit `final_table`. This method distributes computation across an explicitly generated workflow.

C.3 Inference Cost

Table 5 reports average cost on the common 1,152-example coverage subset. Counts are summed across all calls needed to produce a final answer. Gemini’s total-token accounting includes hidden reasoning tokens and is therefore larger than the visible input-plus-output sum.

C.4 Implementation Details

Llama and Qwen are served with vLLM on two H200 nodes with 80GB GPUs each. GPT-4.1 is accessed through the OpenAI Batch API and Gemini-2.5-Flash through Google AI Studio. Single-call non-Gemini methods have a maximum completion budget up to 15,000 tokens, with smaller task-

specific budgets of 2,000 for ZERO SHOT and 4,000 for GUIDED COT. Gemini permits up to 32,000 tokens to account for hidden-token use. LEAST-TO-MOST uses separate 10,000-token planner and 15,000-token executor budgets.

Retry budgets range from three to five depending on the baseline. A generation is retried if JSON cannot be parsed, requested columns are missing or reordered, row lengths disagree with the schema, or a method-specific field is absent. These retries correct output-contract failures, not incorrect cricket reasoning. SC TABLE VOTING always requests five valid candidates before aggregation.

D Controlled Factors and Pressure Construction

D.1 Skill Families

The controlled split labels each example along five families. These labels are derived from the executable query and generation metadata rather than assigned from model outputs.

Entity and schema grounding. *Schema-only* examples contain no grounded batter or bowler variable and depend only on table-level structure. *Single-entity* examples ground exactly one batter or bowler variable. *Multi-entity* examples jointly ground two or more such entities. These labels represent entity-resolution burden rather than broad semantic topic.

Context scoping. *Full-span* examples may draw evidence from anywhere in the complete match narrative. *Partial-span* examples limit reasoning to the first or second half of the context. The distinction concerns search scope, not the type of cricket event.

Temporal reasoning. *Non-temporal controls* contain no explicit time predicate. *Point-based* filters use one-sided relations such as before, before-or-at, after, or after-or-at. *Range-based* filters select a bounded interval. *Interval-overlap* questions ask whether queried and target intervals intersect.

Query operations. *Simple retrieval* includes direct selection, top- k , and ordered retrieval. *Filtered aggregation* aggregates after row filtering without grouping. *Grouped aggregation* computes per-group summaries. *Group comparison* applies post-aggregation filtering, ranking, or comparison over groups.

Model	Baseline	Calls	Input tokens	Output tokens	Total tokens
Llama-3.3-70B	ZERO SHOT	1.0	9424.16	109.52	9533.68
	ZERO-SHOT CoT	1.0	9918.69	323.42	10242.11
	GUIDED CoT	1.0	10371.28	448.74	10820.02
	RowCoT	1.0	10406.29	835.23	11241.51
	PLANFILL	2.0	17913.29	538.29	18451.58
	SC TABLE VOTING	5.0	48126.69	1434.42	49561.11
	LEAST-TO-MOST	3.4	25284.21	1282.26	26566.47
Qwen2.5-72B	ZERO SHOT	1.0	9621.13	112.23	9733.37
	ZERO-SHOT CoT	1.0	10009.04	244.13	10253.17
	GUIDED CoT	1.0	10012.25	378.96	10391.21
	RowCoT	1.0	10358.53	665.64	11024.17
	PLANFILL	2.0	19831.94	484.90	20316.84
	SC TABLE VOTING	5.0	48638.65	1015.57	49654.22
	LEAST-TO-MOST	3.2	27347.44	1454.55	28801.99
GPT-4.1	ZERO SHOT	1.0	9356.42	76.42	9432.84
GPT-4.1	GUIDED CoT	1.0	9850.01	294.62	10144.63
Gemini-2.5-Flash	ZERO SHOT	1.0	9390.76	125.88	21361.03
Gemini-2.5-Flash	GUIDED CoT	1.0	9980.57	595.93	18087.22

Table 5: Average inference cost per example on a shared 1,152-example coverage subset. Multi-call methods sum tokens over all component calls.

Compositional depth. Executable operation chains are binned as 2 steps, 3–4 steps, 5–6 steps, or at least 7 steps. Depth counts the normalized sequence of retrieval, filtering, grouping, aggregation, comparison, and output operations needed to execute the query.

D.2 Pressure Axes

Table 7 defines the mappings used in the main analysis. Grounding is a direct collapse of its factor family. Execution combines query operation and compositional depth. Output combines answer width and height. Localization combines explicit temporal restriction, span restriction, and evidence sparsity.

Pressure axes deliberately compress a larger annotation space. For example, two examples with high localization pressure may reach that label through different combinations of a narrow temporal window and sparse evidence. We therefore use pressure contrasts to summarize recurring difficulty, and factor contrasts to retain interpretability. Neither level should be read as an experimentally randomized intervention.

E Statistical Analysis and Diagnostics

E.1 Paired and Stratified Comparisons

The headline model comparison is paired by example. Confidence intervals are estimated over paired differences, and sign-flip permutation tests evaluate whether the mean paired difference is centered at zero. This preserves the correspondence between model outputs on the same question.

Pressure analyses compare levels within a model. Pooled contrasts use all examples at each level and are useful for descriptive coverage, but may mix different templates and skill combinations. Stratified contrasts match on the remaining observed factors so that the target axis changes within a more comparable condition. A stratum contributes only when both compared levels are represented.

E.2 Balanced Resampling

Support is unequal for several pressure levels. In a pooled contrast, the larger group is repeatedly sampled without replacement down to the size of the smaller group. In a stratified contrast, equalization occurs separately inside each matched stratum before stratum-level differences are combined. Pooled analyses use 2,000 balanced resamples and stratified analyses use 1,000.

The reported balanced interval is the empirical

Family	Skill	Representative question
Grounding	Single entity	For each batter who hit at least one boundary and scored at least one run off Faheem, what percentage of their runs against him came from boundaries?
Grounding Context	Multi-entity Partial span	How many deliveries did Verreynne face while batting against Hasan Ali? Calculate Markram’s strike rate in the specified half of the match for spans in which he faced at least ten balls.
Temporal Temporal Query operation	Point based Range based Grouped aggregation	List wickets claimed by each bowler against each batter before over 29.5. Provide wickets taken by each bowler between overs 2.3 and 32.1. Compare each batter’s strike rate against Shaheen Shah Afridi on boundary-scoring and non-boundary deliveries.
Query operation	Group comparison	For each batter, what percentage of runs against a bowler came from boundaries?
Composition	7+ steps	For each batter, what is the longest streak of consecutive dot balls against Faheem?

Table 6: Representative controlled-split questions. Names and temporal values are grounded variables that change across instantiations.

Axis	Score construction	Level mapping
Grounding	Schema-only = 0; single-entity = 1; multi-entity = 2.	Low = 0; medium = 1; high = 2.
Localization	+1 for temporal reasoning; +1 for a first-half or second-half span; +1 when contributing-row fraction is in (.10, .25]; +2 when it is at most .10. Evidence-count buckets are used when the fraction is unavailable.	Low $\leq$ 1; medium = 2 or 3; high $\geq$ 4.
Execution	Query operation: retrieval = 0, filtered aggregation = 1, grouped aggregation = 2, group comparison = 3. Composition: 2 steps = 0, 3–4 = 1, 5–6 = 2, 7+ = 3. Scores are summed.	Low $\leq$ 1; medium = 2 or 3; high $\geq$ 4.
Output	Columns: 1 = 0, 2–3 = 1, 4–6 = 2, 7+ = 3. Rows: 0–1 = 0, 2–5 = 1, 6–10 = 2, 11–20 = 3, 21+ = 4. Missing counts default to 1. Scores are summed.	Low $\leq$ 1; medium = 2 or 3; high $\geq$ 4.

Table 7: Construction of the controlled-split pressure axes. Contributing-row fraction is the number of rows used in the answer divided by the number scanned.

2.5–97.5 percentile range of these repeated support-balanced estimates. It is not an ordinary bootstrap confidence interval. Its purpose is to test whether the direction of a contrast survives removal of sample-count imbalance by design. Resampling cannot repair absent overlap. Contrasts with only one or two matched strata are retained for auditability but are not used for main conclusions.

We do not interpret the full diagnostic surface as a confirmatory multiple-testing exercise. Main claims require agreement among the raw estimate, paired or sign-flip test, and balanced sensitivity analysis. Isolated threshold crossings in the appendix are descriptive.

E.3 Headline Pressure Contrasts

E.4 Full Pooled Event Contrasts

The pooled table shows large apparent effects for localization and execution, but also extreme execution imbalance: medium-versus-low support differs by more than 72:1. This is why the main text privi-

leges matched-stratum contrasts. After matching, high-versus-medium execution remains stable for both models, while medium-versus-low has only one overlapping stratum and cannot support a broad conclusion.

E.5 Full Stratified Event Contrasts

E.6 Full Pooled Numeric Contrasts

Several pooled localization and output trends appear to suggest lower RMSE at higher pressure. We do not interpret these as evidence that harder localization or larger tables improve computation. Numeric evaluation is conditional on outputs that can be parsed and aligned. Under difficult conditions, the surviving evaluable subset may be systematically easier, creating a survivor effect. Scale differences among query types can also affect RMSE. Matched analyses and directional bias provide a more cautious view.

Factor	Contrast	Event Llama	Event Qwen	Numeric Llama	Numeric Qwen
Temporal	Point-based vs. non-temporal	-8.7*	-2.4	4.6*	3.7*
Temporal	Range-based vs. non-temporal	-9.1*	-3.0	0.5	3.3
Entity grounding	Multi-entity vs. single-entity	-1.3	6.5	12.4*	12.0*
Query operation	Group comparison vs. grouped aggregation	0.6	-3.1	-4.5	0.2
Composition	5–6 steps vs. 3–4 steps	-5.2*	-2.7	9.4*	7.4

Table 8: Selected controlled-split factor contrasts. Event columns report changes in PK F1, where negative is harder; numeric columns report changes in weighted RMSE, where positive is harder. Asterisks mark bootstrap intervals excluding zero.

Model	Axis	Contrast	Raw Δ	Perm. p	Balanced Δ
Llama	Grounding	medium vs. low	12.0	.020	17.1
Qwen	Grounding	medium vs. low	15.2	.005	18.8
Llama	Execution	high vs. medium	-4.5	.042	-4.4
Qwen	Execution	high vs. medium	-6.9	.002	-6.9
Llama	Output	medium vs. low	-12.1	<.001	-10.2
Qwen	Output	medium vs. low	-9.4	.001	-7.6

Table 9: Robust stratified event-identification pressure contrasts. Negative deltas indicate that the first level is harder.

Model	Axis	Contrast	Raw Δ	Perm. p	Balanced Δ
Llama	Grounding	high vs. medium	19.6	.002	11.8
Qwen	Grounding	high vs. medium	8.9	.131	8.8
Llama	Execution	high vs. low	29.9	.032	28.3
Qwen	Execution	high vs. low	43.7	.016	38.3
Llama	Output	medium vs. low	7.8	.027	13.7
Qwen	Output	medium vs. low	2.7	.577	11.3

Table 10: Stratified numeric-fidelity pressure contrasts using weighted RMSE. Positive deltas indicate that the first level has higher error. Qwen grounding and output rows are sensitivity results.

E.7 Full Stratified Numeric Contrasts

F Numeric Bias and Self-Consistency

Table 15 supplies the values behind Figure 3. The declining RMSE at high output pressure should not be confused with improved unconditional performance, because the metric is computed only on aligned and evaluable outputs. Directional rates show that the character of the remaining numeric errors changes sharply. Qwen moves from an 18.8-point overcount bias at low pressure to a 24.9-point undercount bias at high pressure.

Self-consistency achieves substantial agreement without the best numerical accuracy. The final voted Llama tables have average cell support of 3.83 out of five, and 85.9% of samples have mean support above three. Nonetheless, SC TABLE VOTING has RMSE 18.12, compared with 17.67 for ZERO-SHOT CoT and 16.16 for RowCoT. Repeated candidates often reproduce the same arithmetic or evidence-selection error. Voting is effective when candidate noise is independent enough for the correct value to become a plurality; it can-

not repair a systematic computation shared by the model’s samples.

G Recommended Extensions

Three experiments would most directly extend the current evidence. First, a model should generate SQL or code over the gold event table. This measures program induction and computation under privileged structural access. Second, a model should extract its own event table and then generate SQL or code over that table. Comparing the two settings with direct narrative-to-table generation separates extraction, program synthesis, and execution errors. Specialized Text-to-Table systems adapted to query-conditioned answer tables would further clarify how much benefit comes from architectures designed for structured generation.

Third, evaluation should be repeated on at least one non-cricket narrative domain with a different latent record structure. Financial filings, clinical timelines, legal case histories, and incident reports are plausible candidates, but each carries distinct annotation, licensing, and privacy constraints. A multi-domain release should retain domain-specific diagnostics rather than assuming that one shared score captures every form of narrative structure.

Construction provenance should also become an explicit analysis variable. Human-authored, model-generated, rewritten, repaired, and naturally occurring questions can be evaluated as separate subsets. Cross-generator tests—for example, evaluating models unrelated to the construction models—would help determine whether stylistic artifacts contribute to observed performance. These extensions are necessary before Q-STR can support claims beyond its present role as a controlled cricket diagnostic.

Model	Axis	Contrast	Imbalance	Raw Δ	Perm. p	Balanced Δ
Llama	Grounding	medium vs. low	1.175	7.0	<.001	7.0
Llama	Grounding	high vs. low	8.703	-5.7	.018	-5.7
Llama	Grounding	high vs. medium	10.227	-12.7	<.001	-12.8
Llama	Localization	medium vs. low	2.025	-5.6	<.001	-5.6
Llama	Localization	high vs. low	4.907	-19.6	<.001	-19.6
Llama	Localization	high vs. medium	9.938	-13.9	<.001	-14.0
Llama	Execution	medium vs. low	72.500	23.2	<.001	23.2
Llama	Execution	high vs. low	85.938	19.5	.008	19.5
Llama	Execution	high vs. medium	1.185	-3.7	<.001	-3.7
Llama	Output	medium vs. low	1.119	-3.3	.003	-3.3
Llama	Output	high vs. low	3.595	-2.5	.184	-2.5
Llama	Output	high vs. medium	4.024	0.8	.578	0.8
Qwen	Grounding	medium vs. low	1.175	9.4	<.001	9.5
Qwen	Grounding	high vs. low	8.703	-3.1	.218	-3.1
Qwen	Grounding	high vs. medium	10.227	-12.5	<.001	-12.6
Qwen	Localization	medium vs. low	2.025	-5.0	<.001	-5.0
Qwen	Localization	high vs. low	4.907	-18.1	<.001	-18.1
Qwen	Localization	high vs. medium	9.938	-13.0	<.001	-13.1
Qwen	Execution	medium vs. low	72.500	36.1	<.001	36.0
Qwen	Execution	high vs. low	85.938	31.6	<.001	31.6
Qwen	Execution	high vs. medium	1.185	-4.5	<.001	-4.5
Qwen	Output	medium vs. low	1.119	0.5	.658	0.5
Qwen	Output	high vs. low	3.595	-2.2	.253	-2.2
Qwen	Output	high vs. medium	4.024	-2.7	.053	-2.7

Table 11: Pooled event-stage pressure contrasts using PK F1. Imbalance is the ratio of support sizes before balancing. These rows are descriptive because pooled levels can contain different template mixtures.

Model	Axis	Contrast	Strata	Raw Δ	Perm. p	Balanced Δ
Llama	Grounding	medium vs. low	31	12.0	.020	17.1
Llama	Grounding	high vs. low	2	-3.9	1.000	-3.8
Llama	Grounding	high vs. medium	26	-13.2	.116	-4.0
Llama	Localization	medium vs. low	59	-3.8	.170	-5.7
Llama	Localization	high vs. medium	14	-1.9	.679	-5.7
Llama	Execution	medium vs. low	1	-100.0	1.000	-100.0
Llama	Execution	high vs. medium	107	-4.5	.042	-4.4
Llama	Output	medium vs. low	81	-12.1	<.001	-10.2
Llama	Output	high vs. medium	20	3.2	.375	3.4
Qwen	Grounding	medium vs. low	31	15.2	.005	18.8
Qwen	Grounding	high vs. low	2	-19.7	1.000	-20.1
Qwen	Grounding	high vs. medium	26	-0.9	.885	3.6
Qwen	Localization	medium vs. low	59	-3.3	.176	-5.1
Qwen	Localization	high vs. medium	14	-3.6	.607	-3.9
Qwen	Execution	medium vs. low	1	-100.0	1.000	-100.0
Qwen	Execution	high vs. medium	107	-6.9	.002	-6.9
Qwen	Output	medium vs. low	81	-9.4	.001	-7.6
Qwen	Output	high vs. medium	20	-1.6	.755	-4.1

Table 12: Stratified event-stage pressure contrasts. Rows with one or two matched strata are shown for transparency but are not interpreted.

Model	Axis	Contrast	Imbalance	Raw Δ	Perm. p	Balanced Δ
Llama	Grounding	medium vs. low	1.223	6.6	<.001	6.5
Llama	Grounding	high vs. low	4.901	10.7	<.001	10.6
Llama	Grounding	high vs. medium	5.996	4.1	.041	4.1
Llama	Localization	medium vs. low	2.196	-1.5	.182	-1.4
Llama	Localization	high vs. low	3.209	-8.7	<.001	-8.8
Llama	Localization	high vs. medium	7.047	-7.3	<.001	-7.3
Llama	Execution	medium vs. low	1.886	1.7	.155	1.6
Llama	Execution	high vs. low	2.295	11.1	<.001	11.2
Llama	Execution	high vs. medium	1.217	9.5	<.001	9.5
Llama	Output	medium vs. low	1.440	1.5	.166	1.4
Llama	Output	high vs. low	5.756	-5.4	.001	-5.4
Llama	Output	high vs. medium	3.997	-6.9	<.001	-6.9
Qwen	Grounding	medium vs. low	1.223	6.7	<.001	6.7
Qwen	Grounding	high vs. low	4.901	9.1	<.001	9.1
Qwen	Grounding	high vs. medium	5.996	2.4	.275	2.4
Qwen	Localization	medium vs. low	2.196	-1.3	.230	-1.3
Qwen	Localization	high vs. low	3.209	-11.6	<.001	-11.6
Qwen	Localization	high vs. medium	7.047	-10.2	<.001	-10.3
Qwen	Execution	medium vs. low	1.886	0.8	.362	0.8
Qwen	Execution	high vs. low	2.295	12.6	<.001	12.6
Qwen	Execution	high vs. medium	1.217	11.8	<.001	11.8
Qwen	Output	medium vs. low	1.440	1.8	.102	1.8
Qwen	Output	high vs. low	5.756	-3.9	.007	-3.9
Qwen	Output	high vs. medium	3.997	-5.6	.004	-5.7

Table 13: Pooled numeric-stage pressure contrasts using weighted RMSE. Positive values indicate greater error in the first level.

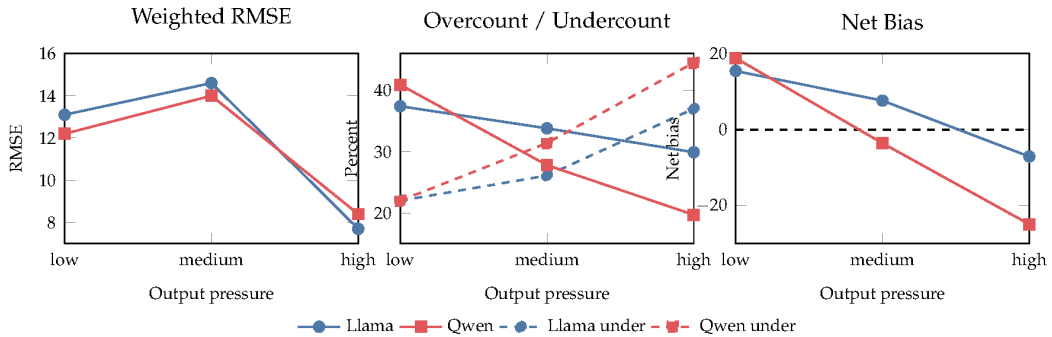


Figure 3: Numeric error profile by output pressure. Both models shift from overcounting toward undercounting as answer-table size increases; the shift is sharper for Qwen.

Model	Axis	Contrast	Strata	Raw Δ	Perm. p	Balanced Δ
Llama	Grounding	medium vs. low	42	2.0	.604	1.8
Llama	Grounding	high vs. low	4	-3.8	.613	-8.1
Llama	Grounding	high vs. medium	33	19.6	.002	11.8
Llama	Localization	medium vs. low	63	-1.4	.586	1.6
Llama	Localization	high vs. medium	25	2.8	.318	1.4
Llama	Execution	medium vs. low	9	19.0	.156	17.9
Llama	Execution	high vs. low	8	29.9	.032	28.3
Llama	Execution	high vs. medium	105	-0.2	.950	-1.3
Llama	Output	medium vs. low	77	7.8	.027	13.7
Llama	Output	high vs. medium	20	0.7	.944	-0.7
Qwen	Grounding	medium vs. low	42	4.1	.132	3.6
Qwen	Grounding	high vs. low	4	3.0	.252	2.7
Qwen	Grounding	high vs. medium	33	8.9	.131	8.8
Qwen	Localization	medium vs. low	63	0.1	.965	0.2
Qwen	Localization	high vs. medium	25	-1.8	.852	-0.4
Qwen	Execution	medium vs. low	9	15.0	.249	13.8
Qwen	Execution	high vs. low	8	43.7	.016	38.3
Qwen	Execution	high vs. medium	105	9.3	.009	5.7
Qwen	Output	medium vs. low	77	2.7	.577	11.3
Qwen	Output	high vs. medium	20	-2.8	.015	-3.0

Table 14: Stratified numeric-stage pressure contrasts using weighted RMSE. Raw and balanced estimates can differ when support remains uneven inside matched conditions.

Model	Output	RMSE	Over	Under	Net
Llama	low	13.1	37.4	22.0	15.4
Llama	medium	14.6	33.8	26.1	7.6
Llama	high	7.7	29.9	37.0	-7.1
Qwen	low	12.2	40.9	22.0	18.8
Qwen	medium	14.0	27.8	31.4	-3.6
Qwen	high	8.4	19.7	44.5	-24.9

Table 15: Exact numeric-error profile by output pressure. Over and Under are percentages; Net is Over minus Under.

Average cell support	Samples	Ratio (%)
[1, 2)	17	0.40
[2, 3)	359	8.43
[3, 4)	1966	46.17
[4, 5)	1694	39.78
= 5	203	4.77

Table 16: Distribution of average cell-level vote support for Llama SC TABLE VOTING, measured over five candidates.